MIT 212 — Programming Techniques

Final Examination Question Bank • Weeks 8–10: Recursion & Data Structures

අවසන් විභාග ප්‍රශ්න බැංකුව — ප්‍රත්‍යාවර්තනය සහ දත්ත ව්‍යුහ (මාදිලි පිළිතුරු සමඟ)

Sections: A. Multiple Choice (20) • B. True/False with corrections (12) • C. Definitions & short answers (10) • D. Comparison questions (6) • E. Code-output questions (10) • F. Error identification & debugging (6) • G. Algorithm / pseudocode (5) • H. Flowchart questions (3) • I. Short programming questions (8) • J. One substantial practical problem. Model answers are given in green boxes after each question.

කොටස්: A. බහුවරණ (20) • B. සත්‍ය/අසත්‍ය නිවැරදි කිරීම් සමඟ (12) • C. අර්ථදැක්වීම් සහ කෙටි පිළිතුරු (10) • D. සංසන්දන ප්‍රශ්න (6) • E. කේත ප්‍රතිදාන (10) • F. දෝෂ හඳුනාගැනීම (6) • G. ඇල්ගොරිතම/ව්‍යාජ කේත (5) • H. ගැලීම් සටහන් (3) • I. කෙටි ක්‍රමලේඛන (8) • J. විශාල ප්‍රායෝගික ගැටලුවක්. සෑම ප්‍රශ්නයකටම පසුව කොළ පැහැති කොටුවේ මාදිලි පිළිතුර ඇත.

Section A — Multiple-Choice Questions බහුවරණ ප්‍රශ්න

Choose the ONE best answer.
වඩාත්ම නිවැරදි පිළිතුර එකක් තෝරන්න.

Q1. Every recursive function must have:
සෑම ප්‍රත්‍යාවර්තී ශ්‍රිතයකටම තිබිය යුත්තේ:

a) Only a recursive case
b) Only a base case
c) A base case AND a recursive case
d) A while loop

> Answer: c) A base case AND a recursive case
> මූලික අවස්ථාව සහ ප්‍රත්‍යාවර්තී අවස්ථාව යන දෙකම.

Q2. factorial(5) returns:
factorial(5) ලබා දෙන අගය:

a) 25
b) 120
c) 15
d) 720

> Answer: b) 120 (5 × 4 × 3 × 2 × 1)

Q3. Python function calls are managed using a:
Python ශ්‍රිත කැඳවීම් කළමනාකරණය වන්නේ:

a) Queue
b) Graph
c) Call stack
d) Linked list

> Answer: c) Call stack — last call pushed finishes first (LIFO).
> call stack (LIFO) මගිනි.

Q4. A stack follows the principle:
ස්ටැක් අනුගමනය කරන මූලධර්මය:

a) FIFO
b) LIFO
c) LILO
d) Random access

> Answer: b) LIFO — Last In, First Out.

Q5. In Python, the correct pair of stack operations on a list is:

a) insert(0,x) / pop(0)
b) append(x) / pop()
c) add(x) / remove()
d) push(x) / popleft()

Answer: b) append(x) adds to the top; pop() removes from the top.

Q6. Which structure should serve help-desk tickets in arrival order?
පැමිණි අනුපිළිවෙලින් ticket සේවා කිරීමට සුදුසු ව්‍යුහය:

a) Stack
b) Queue
c) Binary tree
d) Set

Answer: b) Queue (FIFO) — first ticket submitted is served first.
පෝලිම (FIFO).

Q7. collections.deque method that dequeues (removes the front) is:

a) pop()
b) popleft()
c) remove()
d) dequeue()

Answer: b) popleft()

Q8. In a singly linked list, each node stores:
තනි සම්බන්ධිත ලැයිස්තුවක node එකක ගබඩා වන්නේ:

a) Only data
b) Data + reference to the next node
c) Data + references to next and previous
d) An index number

Answer: b) Data + a reference (pointer) to the next node.
දත්ත + ඊළඟ node වෙත යොමුවක්.

Q9. The first node of a linked list is called the:

a) Root
b) Head
c) Top
d) Front

Answer: b) Head. (Root is for trees; top is for stacks.)

Q10. Compared with an array/Python list, a linked list is better at:

a) Direct access by index
b) Insertion at the front without shifting
c) Using less memory
d) Sorting automatically

Answer: b) Insertion at the front — no elements need shifting; but it has no direct index access.

Q11. A node with no children in a tree is a:
දරුවන් නොමැති tree node එකක් හඳුන්වන්නේ:

a) Root
b) Leaf
c) Sibling
d) Edge

> Answer: b) Leaf.
> පත්‍රය (leaf).

Q12. In a binary tree, each node has at most:

a) 1 child
b) 2 children
c) 3 children
d) Unlimited children

> Answer: b) 2 children — the left child and the right child.

Q13. The BST rule states that for every node:
BST රීතිය:

a) Left subtree larger, right smaller
b) Left subtree smaller, right larger
c) Both subtrees equal
d) Values are unsorted

> Answer: b) Everything in the left subtree is smaller; everything in the right subtree is larger.
> වම් උප වෘක්ෂය කුඩා, දකුණු උප වෘක්ෂය විශාල.

Q14. Which traversal prints BST values in sorted order?
BST අගයන් අනුපිළිවෙලින් මුද්‍රණය කරන ගමන් ක්‍රමය:

a) Pre-order
b) In-order
c) Post-order
d) Level-order

> Answer: b) In-order (Left → Visit → Right).

Q15. Level-order traversal of a tree uses a:

a) Stack
b) Queue
c) Set only
d) Recursion only

> Answer: b) Queue — visit a node, then enqueue its children (breadth-first).

Q16. For the tree with root 1, left child 2, right child 3, the post-order result is:

a) 1, 2, 3

b) 2, 1, 3
c) 2, 3, 1
d) 3, 2, 1

> Answer: c) 2, 3, 1 (Left → Right → Visit).

Q17. Unlike a tree, a graph may contain:
වෘක්ෂයකට වඩා ප්‍රස්තාරයක තිබිය හැක්කේ:

a) A root
b) Nodes
c) Cycles
d) Edges

> Answer: c) Cycles — a path can return to where it started; trees never loop.
> චක්‍ර (cycles).

Q18. A Python dictionary mapping each node to a list of its neighbours is an:

a) Adjacency matrix
b) Adjacency list
c) Edge table
d) Index array

> Answer: b) Adjacency list — compact when there are few edges; the usual choice.

Q19. BFS finds the shortest path (in steps) in an unweighted graph because it:
BFS අවම පියවර මාර්ගය සොයන්නේ:

a) Goes deep first
b) Explores level by level using a queue
c) Uses recursion
d) Sorts the nodes

> Answer: b) It explores level by level with a queue, so nearer nodes are always reached first.
> පෝලිමක් යොදාගෙන මට්ටමෙන් මට්ටම ගවේෂණය කරන නිසාය.

Q20. The visited set in BFS/DFS is needed to:
BFS/DFS හි visited කට්ටලය අවශ්‍ය වන්නේ:

a) Sort output
b) Prevent revisiting nodes and looping forever on cycles
c) Save memory
d) Find the largest node

> Answer: b) Prevent revisiting nodes — without it, cycles cause an infinite loop.
> චක්‍ර නිසා අනන්ත ලූප වීම වැළැක්වීමට.

Section B — True/False (correct the false statements) සත්‍ය/අසත්‍ය — අසත්‍ය ප්‍රකාශ නිවැරදි කරන්න

Q1. A recursive case calls the same function with a smaller or simpler input.

> Answer: TRUE.

Q2. A recursive function without a base case will eventually stop on its own.
මූලික අවස්ථාවක් නැති ප්‍රත්‍යාවර්තී ශ්‍රිතයක් ඉඛේම නවතී.

> Answer: FALSE — it never stops moving toward an end; Python raises RecursionError (stack overflow).
> අසත්‍යයි — Python විසින් RecursionError දෙයි.

Q3. Naive recursive Fibonacci is slow because it repeats the same sub-calculations many times.

> Answer: TRUE — e.g. fib(3) is recomputed repeatedly; memoization fixes this.

Q4. In a stack, the oldest item is removed first.
ස්ටැක් එකක පැරණිතම අයිතමය මුලින්ම ඉවත් වේ.

> Answer: FALSE — the NEWEST item is removed first (LIFO). Oldest-first is a queue.
> අසත්‍යයි — අලුත්ම අයිතමය මුලින් ඉවත් වේ (LIFO).

Q5. stack[-1] reads the top of a list-based stack without removing it (peek).

> Answer: TRUE.

Q6. A queue uses enqueue to add at the front and dequeue to remove from the rear.

> Answer: FALSE — enqueue adds at the REAR; dequeue removes from the FRONT.
> අසත්‍යයි — enqueue පිටුපසට; dequeue ඉදිරියෙන්.

Q7. In a linked list, nodes must sit next to each other in memory.
සම්බන්ධිත ලැයිස්තුවක node මතකයේ එකිනෙක අසල තිබිය යුතුය.

> Answer: FALSE — unlike arrays, nodes can be anywhere; the next references connect them.
> අසත්‍යයි — යොමු (references) මගින් සම්බන්ධ වේ.

Q8. A doubly linked list needs more memory per node than a singly linked list.

> Answer: TRUE — each node stores two links (prev and next).

Q9. The depth of a node is the longest path from that node down to a leaf.

> Answer: FALSE — that is the HEIGHT. Depth is the distance from the ROOT to the node.
> අසත්‍යයි — එය height යි; depth යනු root සිට ඇති දුරයි.

Q10. In-order traversal visits: node → left → right.

> Answer: FALSE — in-order is left → node → right. Node-first is PRE-order.
> අසත්‍යයි — in-order: වම → node → දකුණ.

Q11. A tree is a special kind of graph that is connected and has no cycles.

> Answer: TRUE.

Q12. DFS uses a queue, and BFS uses a stack.
DFS පෝලිමක් ද BFS ස්ටැක් එකක් ද භාවිතා කරයි.

> Answer: FALSE — it is the reverse: BFS → queue; DFS → recursion or a stack.
> අසත්‍යයි — BFS → පෝලිම; DFS → ප්‍රත්‍යාවර්තනය/ස්ටැක්.

Section C — Definitions & Short Answers අර්ථදැක්වීම් සහ කෙටි පිළිතුරු

Q1. Define recursion and name its two essential parts.
ප්‍රත්‍යාවර්තනය අර්ථ දක්වා එහි අත්‍යවශ්‍ය කොටස් දෙක නම් කරන්න.

> Answer: Recursion is a technique where a function solves a problem by calling itself with a smaller version of the same problem. Its two parts: the base case (returns without calling itself, stopping the recursion) and the recursive case (calls the function with smaller input).
> ශ්‍රිතයක් එකම ගැටලුවේ කුඩා අනුවාදයක් සමඟ තමන්වම කැඳවමින් විසඳන ක්‍රමයයි. කොටස්: මූලික අවස්ථාව (නතර වීම) සහ ප්‍රත්‍යාවර්තී අවස්ථාව (කුඩා ආදානයෙන් නැවත කැඳවීම).

Q2. What is the call stack and how does it behave during factorial(4)?

> Answer: The call stack stores active function calls in LIFO order. factorial(4) pushes calls for 4, 3, 2, 1; when factorial(1) returns 1, the stack unwinds: 2×1=2, 3×2=6, 4×6=24.

Q3. Define stack and queue in one sentence each, naming their removal rules.
ස්ටැක් සහ පෝලිම වාක්‍ය එක බැගින් අර්ථ දක්වන්න.

> Answer: A stack is a LIFO structure where the newest item is removed first (push/pop at the top). A queue is a FIFO structure where the oldest item is removed first (enqueue at rear, dequeue at front).
> ස්ටැක් = LIFO (අලුත්ම දෙය මුලින් ඉවත්); පෝලිම = FIFO (පැරණිතම දෙය මුලින් ඉවත්).

Q4. What is a node in a linked list? Write its Python class.

> Answer: A node is a small object holding data and a reference to the next node: class Node: def __init__(self, data): self.data = data; self.next = None

Q5. Define head and explain what happens if you lose the head reference.
Head යනු කුමක්ද? එය නැති වුවහොත් කුමක් වේද?

> Answer: The head is the reference to the first node; traversal starts there. Losing it makes the whole list unreachable, since nodes are only reachable by following next links from the head.
> Head යනු පළමු node වෙත යොමුවයි. එය නැති වුවහොත් මුළු ලැයිස්තුවම ළඟා විය නොහැක.

Q6. Define: root, leaf, siblings, subtree.

> Answer: Root — the single top node of a tree. Leaf — a node with no children. Siblings — nodes sharing the same parent. Subtree — any node together with all of its descendants.

Q7. State the Binary Search Tree (BST) rule and why it makes searching fast.
BST රීතිය සහ සෙවීම වේගවත් වන හේතුව.

> Answer: For every node, all values in its left subtree are smaller and all values in its right subtree are larger. Each comparison discards half of the remaining nodes, so even a huge tree is searched in very few steps.
> සෑම node එකකටම වමේ අගයන් කුඩා, දකුණේ විශාල. සෑම සංසන්දනයක්ම ඉතිරි node වලින් අඩක් ඉවත් කරන නිසා සෙවීම ඉතා වේගවත්ය.

Q8. What is graph traversal and give two real questions it answers.

Answer: Traversal means systematically visiting a graph's nodes by following edges. It answers questions such as "is node B reachable from A?" and "what is the shortest path (fewest steps) between two people/places?"

Q9. Differentiate a directed, undirected, and weighted edge with one example each.

Directed, undirected, weighted දාර වෙන් කර දක්වන්න.

Answer: Undirected: goes both ways (friendship — if A knows B, B knows A). Directed: one way only, A → B (one-way street, web hyperlink). Weighted: carries a number such as distance, cost or time (a road of 70 km).

Undirected: දෙපැත්තටම; Directed: එක් දිශාවකට පමණි; Weighted: දුර/පිරිවැය වැනි අගයක් සහිතයි.

Q10. What are the degrees of separation between two people in a friendship graph, and which algorithm finds it?

Answer: It is the number of BFS levels crossed from one person to the other — i.e. the shortest path in steps. BFS finds it, because BFS reaches nearer nodes first.

Section D — Comparison Questions සංසන්දන ප්‍රශ්න

Q1. Compare a stack and a queue under: removal rule, Python operations, and one real use each.
ස්ටැක් සහ පෝලිම සංසන්දනය කරන්න.

> Answer: Stack: LIFO; list.append() / list.pop(); use: undo/redo, browser back, call stack. Queue: FIFO; deque.append() / deque.popleft(); use: print queue, ticket line, task scheduling, BFS. The difference is not storage — it is the removal rule.
> ස්ටැක්: LIFO, append/pop, Undo/back. පෝලිම: FIFO, append/popleft, ticket/print පෝලිම්. වෙනස ගබඩාව නොව ඉවත් කිරීමේ රීතියයි.

Q2. Compare an array (Python list) with a linked list.
Array සහ linked list සංසන්දනය.

> Answer: Array: fast direct access by index, simple, but middle insert/delete shifts elements; good when size is stable. Linked list: flexible growth, fast insertion when the position is known, no index access, extra memory for references. Rule of thumb: arrays are access-friendly; linked lists are link-manipulation-friendly.
> Array: index මගින් වේගවත් ප්‍රවේශය; මැදට ඇතුළත් කිරීම මන්දගාමී. Linked list: නම්‍යශීලී වර්ධනය; index ප්‍රවේශයක් නැත; යොමු සඳහා අමතර මතකය.

Q3. Compare a tree and a graph (at least four points).
වෘක්ෂය සහ ප්‍රස්තාරය සංසන්දනය (කරුණු 4+).

> Answer: Tree: one root; no cycles; exactly one path between any two nodes; parent→child direction; examples: folders, org charts. Graph: no special root; cycles allowed; many possible paths; edges may have any/no direction; examples: maps, social networks. A tree is a special kind of graph: connected with no cycles.
> වෘක්ෂය: root එකයි, චක්‍ර නැත, මාර්ග එකයි. ප්‍රස්තාරය: root නැත, චක්‍ර ඇත, මාර්ග බොහෝය. වෘක්ෂය යනු චක්‍ර නොමැති විශේෂ ප්‍රස්තාරයකි.

Q4. Compare pre-order, in-order, post-order and level-order traversal: visiting order, structure used, and typical use.

> Answer: Pre-order (Visit→L→R, recursion): copy/save a tree top-down. In-order (L→Visit→R, recursion): prints BST values sorted. Post-order (L→R→Visit, recursion): delete/evaluate bottom-up. Level-order (row by row, QUEUE — BFS): visit nearest levels first.

Q5. Compare BFS and DFS: order, structure, strength, memory, and one use case each.
BFS සහ DFS සංසන්දනය.

> Answer: BFS — level by level; uses a queue (FIFO); finds shortest path in steps; memory wider (holds a whole level); use: nearest friend / fewest hops. DFS — deep down one path then backtrack; uses recursion or a stack; explores full branches; memory deeper (holds one path); use: maze solving, connectivity checks.
> BFS: මට්ටමෙන් මට්ටම, පෝලිම, කෙටිම මාර්ගය. DFS: එක් මාර්ගයක් ගැඹුරට, ප්‍රත්‍යාවර්තනය/ස්ටැක්, අතු සම්පූර්ණ ගවේෂණය.

Q6. Compare the adjacency list and the adjacency matrix for storing a graph.

> Answer: Adjacency list: each node keeps a list of its neighbours (a Python dict); compact when there are few edges — the usual choice; neighbour lookup is instant. Adjacency matrix: an N×N grid of 1/0; instant edge lookup between any two nodes, but uses more space (N^2 cells even for few edges).

Section E — Code-Output Questions කේත ප්‍රතිදාන ප්‍රශ්න

Predict the exact output of each program.
සෑම වැඩසටහනකම නිවැරදි ප්‍රතිදානය පුරෝකථනය කරන්න.

Q1. What is the output?

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
```

> Answer: 24 — the stack unwinds as 1 → 2×1 → 3×2 → 4×6 = 24.
> ප්‍රතිදානය 24.

Q2. What is the output?

```
def fibonacci(n):
    if n == 0: return 0
    if n == 1: return 1
    return fibonacci(n-1) + fibonacci(n-2)

print(fibonacci(6))
```

> Answer: 8 — the sequence is 0, 1, 1, 2, 3, 5, 8: fib(6) = 8.

Q3. What is the output?

```
def mystery(n):
    if n == 0:
        return
    print(n)
    mystery(n - 1)

mystery(3)
```

> Answer: 3, then 2, then 1 (each on its own line). Printing happens BEFORE the recursive call, so it counts down.
> ප්‍රතිදානය: 3 2 1 (පේළි වෙන වෙනම).

Q4. What is the output?

```
stack = []
stack.append('home')
stack.append('news')
stack.append('sports')
stack.pop()
print(stack[-1])
```

> Answer: news — 'sports' was popped; peek (stack[-1]) shows the new top.

Q5. What is the output?

```
from collections import deque
q = deque()
q.append('A'); q.append('B'); q.append('C')
print(q.popleft())
print(q.popleft())
```

> Answer: A then B — popleft() removes from the front in FIFO order.
> A පසුව B (FIFO).

Q6. What is the output?

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

a = Node(10); b = Node(20); c = Node(30)
a.next = b; b.next = c
cur = a
while cur is not None:
    print(cur.data, end=' ')
    cur = cur.next
```

Answer: 10 20 30 — the loop follows next references from the head until None.

Q7. What is the output?

```
def insert_front(head, value):
    new = Node(value)
    new.next = head
    return new

head = insert_front(a, 5)   # a is 10->20->30
print(head.data, head.next.data)
```

Answer: 5 10 — the new node points to the old head and becomes the new head.

Q8. What is the output?

```
def in_order(node):
    if node is None: return
    in_order(node.left)
    print(node.value, end=' ')
    in_order(node.right)

# BST built from 8, 3, 12, 1, 6
in_order(root)
```

Answer: 1 3 6 8 12 — in-order traversal of a BST prints values in sorted order.
BST in-order → අනුපිළිවෙලින්: 1 3 6 8 12.

Q9. What is the output?

```
# same BST: 8 root; 3, 12 children; 1, 6 under 3
level_order(root)   # queue-based
```

Answer: 8 3 12 1 6 — row by row from the top (level-order / BFS on the tree).

Q10. What is the output?

```
graph = {'A': ['B','E'], 'B': ['A','C'], 'C': ['B','D'],
         'D': ['C','E'], 'E': ['A','D']}
bfs(graph, 'A')   # standard queue + visited version
```

Answer: A B E C D — level 0: A; level 1: A's neighbours B, E; level 2: their new neighbours C, D.
A B E C D — මට්ටමෙන් මට්ටම.

Section F — Error Identification & Debugging දෝෂ හඳුනාගැනීම සහ නිවැරදි කිරීම

Each snippet contains at least one error. Identify it, name the error type, and write the correction.
සෑම කේත කැබැල්ලකම අවම දෝෂයක් ඇත. එය හඳුනාගෙන, වර්ගය නම් කර, නිවැරදි කරන්න.

Q1. Find and fix the error(s):
දෝෂ(ය) සොයා නිවැරදි කරන්න:

```
def countdown(n):
    print(n)
    countdown(n - 1)
```

Answer: Missing base case → infinite recursion → RecursionError (runtime error). Fix: add if n == 0: return before the print (or stop when n < 1).
මූලික අවස්ථාව නැත → අනන්ත ප්‍රත්‍යාවර්තනය. නිවැරදි කිරීම: if n == 0: return එක් කරන්න.

Q2. Find and fix the error(s):
දෝෂ(ය) සොයා නිවැරදි කරන්න:

```
def factorial(n):
    if n == 1:
        return 1
    return n * factorial(n)
```

Answer: The input does not shrink — factorial(n) calls factorial(n) again (logical error causing infinite recursion). Fix: factorial(n - 1).
ආදානය කුඩා නොවේ. factorial(n - 1) විය යුතුය.

Q3. Find and fix the error(s):
දෝෂ(ය) සොයා නිවැරදි කරන්න:

```
def add_up(n):
    if n == 0:
        return 0
    n + add_up(n - 1)
```

Answer: Missing return in the recursive case — the result is lost and the function returns None (logical error). Fix: return n + add_up(n - 1).
return නැති වීමෙන් ප්‍රතිඵලය නැති වේ. return n + add_up(n - 1) ලෙස ලියන්න.

Q4. Find and fix the error(s):
දෝෂ(ය) සොයා නිවැරදි කරන්න:

```
queue = []
queue.append('t1'); queue.append('t2')
served = queue.pop()   # serve first ticket
print(served)
```

Answer: Logical error: list.pop() removes the LAST item (t2), so tickets are served LIFO not FIFO. Fix: use collections.deque and popleft(), or queue.pop(0).
pop() අවසාන අයිතමය ඉවත් කරයි (LIFO). deque + popleft() භාවිතා කරන්න.

Q5. Find and fix the error(s):
දෝෂ(ය) සොයා නිවැරදි කරන්න:

```
def insert_front(head, value):
    new_node = Node(value)
    head = new_node
    new_node.next = head
    return new_node
```

Answer: The old head reference is overwritten BEFORE linking, so new_node.next points to itself and the rest of the list is lost. Fix order: new_node.next = head FIRST, then return new_node.
පැරණි head යොමුව සම්බන්ධ කිරීමට පෙර නැති කර ඇත. මුලින් new_node.next = head, පසුව return.

Q6. Find and fix the error(s):
දෝෂ(ය) සොයා නිවැරදි කරන්න:

```
def dfs(graph, node):
    print(node)
    for nbr in graph[node]:
        dfs(graph, nbr)
```

Answer: No visited set — on a graph with cycles (e.g. A–B–A) this recurses forever (RecursionError). Fix: pass a visited set, add each node on entry, and only recurse into neighbours not in visited.
visited කට්ටලයක් නැත — චක්‍ර මත අනන්ත ලූප වේ. visited එක් කර, නොගිය අසල්වැසියන්ට පමණක් යන්න.

Section G — Algorithm / Pseudocode Questions ඇල්ගොරිතම / ව්‍යාජ කේත ප්‍රශ්න

Q1. Write pseudocode for recursive factorial(n).
ප්‍රත්‍යාවර්තී factorial(n) සඳහා ව්‍යාජ කේත ලියන්න.

> Model answer:

```
FUNCTION factorial(n)
    IF n == 1 THEN
        RETURN 1
    ELSE
        RETURN n * factorial(n - 1)
    ENDIF
ENDFUNCTION
```

The IF branch is the base case; the ELSE branch shrinks the problem.

Q2. Write pseudocode for a browser back button using a stack (visit 3 pages, go back once).

> Model answer:

```
history ← empty stack
PUSH 'home' ONTO history
PUSH 'news' ONTO history
PUSH 'sports' ONTO history
POP history            // leave sports
current ← TOP of history
PRINT current          // news
```

The last visited page is revisited first — exactly LIFO.

Q3. Write pseudocode for searching a value v in a BST.
BST එකක v අගය සෙවීමට ව්‍යාජ කේත.

> Model answer:

```
FUNCTION search(node, v)
    IF node IS NULL THEN RETURN NOT FOUND
    IF v == node.value THEN RETURN FOUND
    IF v < node.value THEN
        RETURN search(node.left, v)
    ELSE
        RETURN search(node.right, v)
ENDFUNCTION
```

Each comparison discards half the remaining nodes — e.g. finding 6 in {8,3,12,1,6}: 6<8 go left, 6>3 go right, found in 3 steps.

Q4. Write pseudocode for BFS from a start node.
ආරම්භක node සිට BFS සඳහා ව්‍යාජ කේත.

> Model answer:

```
FUNCTION BFS(graph, start)
    visited ← {start}
    queue ← [start]
    WHILE queue NOT EMPTY
        node ← DEQUEUE front of queue
        VISIT node
        FOR EACH nbr IN graph[node]
            IF nbr NOT IN visited THEN
                ADD nbr TO visited
                ENQUEUE nbr
ENDFUNCTION
```

Queue = level-by-level order; visited prevents looping on cycles.

Q5. Write pseudocode for counting the leaves of a tree recursively.

> Model answer:

```
FUNCTION count_leaves(node)
```

```
    IF node HAS NO children THEN
        RETURN 1
    total ← 0
    FOR EACH child IN node.children
        total ← total + count_leaves(child)
    RETURN total
ENDFUNCTION
```

Base case: a leaf counts as 1. Recursive case: sum the leaf counts of every child subtree.

Section H — Flowchart Questions ගැලීම් සටහන් ප්‍රශ්න

Describe/draw each flowchart using standard symbols: oval = Start/End, parallelogram = Input/Output, rectangle = Process, diamond = Decision.
සම්මත සංකේත භාවිතා කරන්න: ඉලිප්සය = ආරම්භය/අවසානය, සමාන්තරාස්‍රය = ආදාන/ප්‍රතිදාන, සෘජුකෝණාස්‍රය = ක්‍රියාවලිය, දියමන්තිය = තීරණය.

Q1. Draw a flowchart for recursive factorial(n).
factorial(n) සඳහා ගැලීම් සටහන අඳින්න.

> Model answer (text form of the flowchart):

```
(Start) → [Input n]
→ <n == 1 ?>
      Yes → [Return 1] → (End)
      No  → [result = n × factorial(n−1)]
            → [Return result] → (End)
```

The decision diamond is the base-case test; the process box calls the function again with n−1.

Q2. Draw a flowchart for serving all tickets in a queue.
පෝලිමේ සියලු ticket සේවා කිරීමේ ගැලීම් සටහන.

> Model answer (text form of the flowchart):

```
(Start) → [Create empty queue] → [Enqueue tickets]
→ <queue empty ?>
      Yes → (End)
      No  → [Dequeue front ticket] → [Serve / print ticket]
            → back to <queue empty ?>
```

The loop arrow returns to the decision until the queue is empty — a classic FIFO service loop.

Q3. Draw a flowchart for BFS from a start node.
BFS සඳහා ගැලීම් සටහන.

> Model answer (text form of the flowchart):

```
(Start) → [visited = {start}; queue = [start]]
→ <queue empty ?>
      Yes → (End)
      No  → [node = dequeue front] → [Visit node]
            → [For each unvisited neighbour:
               add to visited, enqueue]
            → back to <queue empty ?>
```

Same service-loop shape as the queue flowchart — BFS is a queue-driven loop with a visited set.

Section I — Short Programming Questions කෙටි ක්‍රමලේඛන ප්‍රශ්න

Q1. Write factorial(n) recursively with input validation for n < 1.

n < 1 සඳහා වලංගුතා පරීක්ෂාව සහිත factorial(n).

> Model answer:

```
def factorial(n):
    if n < 1:
        raise ValueError('n must be >= 1')
    if n == 1:
        return 1
    return n * factorial(n - 1)
```

Q2. Write fibonacci(n) recursively, then print the first 8 Fibonacci numbers.

> Model answer:

```
def fibonacci(n):
    if n == 0: return 0
    if n == 1: return 1
    return fibonacci(n - 1) + fibonacci(n - 2)

for i in range(8):
    print(fibonacci(i), end=' ')   # 0 1 1 2 3 5 8 13
```

Q3. Simulate a browser back button: visit 5 pages, then go back two pages, printing the current page after each back action.

Back බොත්තම අනුකරණය කරන්න.

> Model answer:

```
history = ['home', 'news', 'sports', 'mail', 'maps']
for _ in range(2):
    history.pop()
    print('Now on:', history[-1])
# Now on: mail   /   Now on: sports
```

Q4. Use deque to add 5 help-desk tickets and serve the first 3, printing remaining tickets after each service.

> Model answer:

```
from collections import deque
tickets = deque(['T1','T2','T3','T4','T5'])
for _ in range(3):
    served = tickets.popleft()
    print('Served', served, '| waiting:', list(tickets))
```

Q5. Build Node and LinkedList classes; add 10, 20, 30 and print "10 -> 20 -> 30 -> None".

Linked list එකක් සාදා මුද්‍රණය කරන්න.

> Model answer:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None
    def add_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new; return
```

```
        cur = self.head
        while cur.next is not None:
            cur = cur.next
        cur.next = new
    def show(self):
        cur = self.head
        while cur is not None:
            print(cur.data, '->', end=' ')
            cur = cur.next
        print('None')

lst = LinkedList()
for v in (10, 20, 30): lst.add_end(v)
lst.show()   # 10 -> 20 -> 30 -> None
```

Q6. Using the TreeNode class (value + children list), build a root with three children, give one child two children, and write count_leaves(node).

> Model answer:

```
class TreeNode:
    def __init__(self, value):
        self.value = value
        self.children = []
    def add_child(self, child):
        self.children.append(child)

root = TreeNode('CEO')
a, b, c = TreeNode('A'), TreeNode('B'), TreeNode('C')
for m in (a, b, c): root.add_child(m)
a.add_child(TreeNode('A1')); a.add_child(TreeNode('A2'))

def count_leaves(node):
    if not node.children:
        return 1
    return sum(count_leaves(ch) for ch in node.children)

print(count_leaves(root))   # 4  (A1, A2, B, C)
```

Q7. Build the BST 8 / 3,12 / 1,6 with a Node class and print its values in sorted order using in_order.
BST එක සාදා in-order මුද්‍රණය කරන්න.

> Model answer:

```
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

root = Node(8)
root.left = Node(3); root.right = Node(12)
root.left.left = Node(1); root.left.right = Node(6)

def in_order(node):
    if node is None:
        return
    in_order(node.left)
    print(node.value, end=' ')
    in_order(node.right)

in_order(root)   # 1 3 6 8 12
```

Q8. Store a 5-person friendship graph as a dictionary, print each person with their friends and their degree.

Model answer:

```
graph = {
    'Nimal': ['Asha', 'Ravi'],
    'Asha':  ['Nimal', 'Sara'],
    'Ravi':  ['Nimal', 'Sara'],
    'Sara':  ['Asha', 'Ravi', 'Omar'],
    'Omar':  ['Sara'],
}
for person, friends in graph.items():
    print(person, '->', friends, '| degree:', len(friends))
```

Section J — Substantial Practical Problem විශාල ප්‍රායෝගික ගැටලුව

"Campus Help Desk & Friend Finder" console program "Campus Help Desk සහ Friend Finder" වැඩසටහන

The Faculty of Graduate Studies wants a small console system with FOUR parts. This one problem combines every Week 8–9 structure: a stack, a queue, a BST and a graph with BFS.

FGS පීඨයට කොටස් හතරකින් යුත් කුඩා console පද්ධතියක් අවශ්‍යයි. මෙම එක් ගැටලුවෙන් 8–9 සතිවල සියලු ව්‍යුහ ආවරණය වේ: ස්ටැක්, පෝලිම, BST සහ BFS සහිත ප්‍රස්තාරයක්.

- Part 1 (Stack): keep a "recent actions" history. Record 5 actions, then undo the last 2, printing the current latest action after each undo.
  1 කොටස (ස්ටැක්): "මෑත ක්‍රියා" ඉතිහාසය. ක්‍රියා 5ක් සටහන් කර අවසාන 2 undo කරන්න.
- Part 2 (Queue): manage waiting help-desk tickets. Enqueue 4 tickets and serve them all in arrival order, printing each.
  2 කොටස (පෝලිම): ticket 4ක් එකතු කර පැමිණි අනුපිළිවෙලින් සේවා කරන්න.
- Part 3 (BST): insert student IDs 45, 20, 70, 10, 30 into a BST; print them in sorted order; then search for ID 30, reporting the comparisons made.
  3 කොටස (BST): ID අංක ඇතුළත් කර, අනුපිළිවෙලින් මුද්‍රණය කර, 30 සොයන්න.
- Part 4 (Graph + BFS): store the student friendship graph as an adjacency list and use BFS to print the order in which students are reached from "Nimal", and Nimal's degrees of separation from "Omar".
  4 කොටස (ප්‍රස්තාරය + BFS): "Nimal" සිට BFS අනුපිළිවෙල සහ "Omar" දක්වා degrees of separation මුද්‍රණය කරන්න.

Model solution මාදිලි විසඳුම

```
from collections import deque

# ---------- Part 1: Stack (recent actions / undo) ----------
actions = []
for act in ['login', 'open case', 'upload file', 'edit note', 'save']:
    actions.append(act)              # push
print("History:", actions)
for _ in range(2):                   # undo last two
    undone = actions.pop()           # pop = LIFO
    print("Undid:", undone, "| now latest:", actions[-1])

# ---------- Part 2: Queue (help-desk tickets) ----------
tickets = deque(['T1-password', 'T2-wifi', 'T3-email', 'T4-printer'])
while tickets:
    served = tickets.popleft()       # dequeue = FIFO
    print("Serving", served, "| waiting:", list(tickets))

# ---------- Part 3: BST (student IDs) ----------
class Node:
    def __init__(self, value):
        self.value = value
        self.left = None
        self.right = None

def insert(node, value):
    if node is None:
        return Node(value)           # base case: empty spot found
    if value < node.value:
        node.left = insert(node.left, value)
    else:
        node.right = insert(node.right, value)
    return node

root = None
for sid in (45, 20, 70, 10, 30):
    root = insert(root, sid)

def in_order(node):
    if node is None:
```

```
        return
    in_order(node.left)
    print(node.value, end=" ")
    in_order(node.right)

print("Sorted IDs:", end=" "); in_order(root); print()
                                    # 10 20 30 45 70
def search(node, v, steps=0):
    if node is None:
        return None, steps
    steps += 1
    if v == node.value:
        return node, steps
    if v < node.value:
        return search(node.left, v, steps)
    return search(node.right, v, steps)

found, steps = search(root, 30)
print("Found 30 in", steps, "comparisons")   # 45 -> 20 -> 30 = 3

# ---------- Part 4: Graph + BFS ----------
graph = {
    'Nimal': ['Asha', 'Ravi'],
    'Asha':  ['Nimal', 'Sara'],
    'Ravi':  ['Nimal', 'Sara'],
    'Sara':  ['Asha', 'Ravi', 'Omar'],
    'Omar':  ['Sara'],
}

def bfs_order_and_distance(graph, start, target):
    visited = {start}
    q = deque([(start, 0)])          # (node, level)
    order = []
    dist = None
    while q:
        node, level = q.popleft()
        order.append(node)
        if node == target:
            dist = level
        for nbr in graph[node]:
            if nbr not in visited:
                visited.add(nbr)
                q.append((nbr, level + 1))
    return order, dist

order, dist = bfs_order_and_distance(graph, 'Nimal', 'Omar')
print("BFS order:", order)
   # [Nimal, Asha, Ravi, Sara, Omar]
print("Degrees of separation Nimal -> Omar:", dist)   # 3
```

Expected behaviour: Part 1 undoes “save” then “edit note” (LIFO). Part 2 serves T1→T4 in arrival order (FIFO). Part 3 prints 10 20 30 45 70 (in-order = sorted) and finds 30 in 3 comparisons (45→20→30). Part 4 visits Nimal, Asha, Ravi, Sara, Omar level by level and reports 3 degrees of separation (Nimal→Asha→Sara→Omar). Marking focus: correct removal rules, base cases in every recursive function, and the visited set in BFS.

අපේක්ෂිත හැසිරීම: 1 කොටස LIFO ලෙස undo; 2 කොටස FIFO ලෙස සේවා; 3 කොටස 10 20 30 45 70 මුද්‍රණය කර සංසන්දන 3කින් 30 සොයයි; 4 කොටස මට්ටමෙන් මට්ටම ගොස් degrees of separation = 3 ලෙස වාර්තා කරයි. ලකුණු අවධානය: නිවැරදි ඉවත් කිරීමේ රීති, සෑම ප්‍රත්‍යාවර්තී ශ්‍රිතයකම මූලික අවස්ථා, සහ BFS හි visited කට්ටලය.

Good luck, Lasa! Pair this question bank with the Revision Pack: read a sheet, then attempt the matching sections here under exam timing.

සුබ පැතුම්! පුනරීක්ෂණ ඇසුරුම සමඟ මෙම ප්‍රශ්න බැංකුව යොදාගන්න — පත්‍රිකාවක් කියවා අදාළ කොටස් විභාග වේලාවට උත්සාහ කරන්න.